CS2109S Introduction to AI. and ML

AY24/25 Sem 2. github.com/gerteck

- **Search Algorithms** (Uninformed Search (BFS/DFS), Local Search (Hill Climbing), Informed Search (A*), Adversarial Search (Minimax))
- **"Classical" ML** (Decision Trees, Linear/Logistic Regression, Kernels and Support Vector Machines, "Classical" Unsupervised Learning)
- **"Modern" ML** (Neural Networks, Deep Learning, Sequential Data)

1. Introduction to AI

Intelligent Agent and Environments

An agent perceives environment through sensors, act through actuators.
Rational: choose actions to maximize expected performance.

- **Percept**: content agent's sensors are perceiving. **Percept Sequence** complete history of things agent has perceived.
- **PEAS**: Performance Measure (definition of success), Environment (world agent operates in), Actuators (actions agent can perform), Sensors (Inputs agent gets from environment)
  (E.g. Chatbot: Sensor as text input/chat history, Actuators as output, Environment as chat interface, plugins, Perf. measure as correctness safety.)
- **Exploration vs. Exploitation**: Exploration: learn more about world. Exploitation: Maximize gain based on current knowledge.

Properties of Task Environments

Observable: Fully vs. Partially observable.
Deterministic vs. Stochastic: Next state determined by current state and action. If environment also dependent on actions of other agents, also *strategic*, unless other agents are predictable.
Episodic vs. Sequential: Single-step vs. multi-step tasks. (Next episode does/does not depend on actions taken in previous episode.)
Static vs. Dynamic: Environment changes while deliberating. - semi-dynamic (env no change, only change agent performance score)
Discrete vs. Continuous: Finite (distinct, clearly defined) vs. infinite (states/percepts)/actions.
Single-agent vs. Multi-agent: Operating alone or with others.

Common Agent Structures

Simple Reflex Agent: Condition-action rules (e.g., "If up empty: up"). Based on current percept, ignore percept history.
Goal-Based Agent: Acts to achieve goals. Tracks state of world, goals.
Utility-Based Agent: Maximizes utility (e.g., rewards). Use utility function (perf. measure) to assign scores.
Learning Agent: Adapts and improves over time, using performance element, critic and problem generator.

2. Problem Solving by Searching

Search Problem

Search problem defined where goal is to find state/(path to state) from a set of possible states.
State Space. Set of possible states environment can be.
Representation Invariant: Abstract states hv. mapped concrete states.
Initial State where agent starts, **Goal State(s)**: can be > 1, use goal test.
Actions. Given state s, finite set of actions that can be executed.
Transition Model. Describes what each action does. Returns state resulting from action a in state s.
Action Cost Function: Numeric cost of applying action to reach next state.
Search Algorithm: Takes search problem as input, returns a solution / failure. Solution could be sequence of actions, or final state. Optimal solution is one with least cost.

Search Algorithms

Search algorithms explore the state space to find solution. Key operations:
Frontier: Set of nodes to explore next. *Explored Set*: Nodes already visited.
Node: Contains state, parent node, action, path cost, and depth.

Performance/Evaluation
Completeness: Finds solution if exists / report failure for none.
Optimality (Cost) : If outputs solution, it is lowest path cost of all solutions. Algorithm can be both optimal and incomplete.
Time, Space complexity: How long, how much memory needed.

Uninformed Search Algorithms

No information to guide search, aka blind search, no clue how good state is.

Breadth-First Search (BFS)

- **Frontier**: Queue (FIFO). Explore all nodes at curr. depth, before deeper.
- **Complete**: if branching factor b finite. **Optimal**: if step cost uniform.
- **Complexity**: Time: $O(b^d)$, d is depth of shallowest sln. Space: $O(b^d)$.
- *Optimize*: track visited set, early goal test (check node soln once generated).

Uniform-Cost Search (UCS) (Dijkstra's SSSP 'equiv')

- **Frontier**: Priority queue by min total path cost. Expand least-cost first.
- No-visited-mem ('tree-search', as if tree). Visited-mem ('graph-search').
- Even if goal state in frontier, sorted, first g-state selected must be optimal.
- **Complete**: if step costs are positive. **Optimal**: if step costs are positive.
- **Complexity**: Time: $O(b^{C^*/\epsilon})$, C^* is the optimal cost, ϵ is minimum step cost. Space: $O(b^{C^*/\epsilon})$. (w.r.t 'tier' of optimal solution)

Depth-First Search (DFS)

- **Frontier**: Stack (LIFO). Explores as deep as possible before backtrack.
- **Incomplete**: if search tree has infinite depth or loops.
  Non-optimal: No, may find suboptimal solutions.
- **Complexity**: Time: $O(b^m)$, where m is max depth. Space: $O(bm)$.

Depth-Limited Search (DLS)

- Limit search depth to l. Backtrack when limit hit. Good if opt. d known.
- **Incomplete**: if sln exists beyond depth l. **Non-optimal**: if used w. DFS.
- **Complexity**: Time: $O(b^l)$. Space: $O(bl)$.

Iterative Deepening Search (IDS)

- Repeatedly applies DLS with increasing depth limits. Try all possible depth limits from 0 till sln found or failure value returned from DLS. Combines benefits of BFS and DFS.
- **Complete**: If branching factor b finite. **Optimal**: if step costs uniform.
- **Complexity**: Time : $O(b^d)$. Space: $O(bd)$.

Informed Search Algorithms

Guide search with extra info, aka heuristic function on how close to goal.
Heuristic $h(n)$ estimates cost to reach goal from node n. $h^*(n)$: true cost.

- **Admissible**: Never overestimates cost ($h(n) \leq h^*(n)$). (e.g. $h(n) = 0$)
- **Consistent**: if for every node n, every successor n' of n generated by any action a: $h(n) \leq c(n, a, n') + h(n')$ and $h(G) = 0$. Δ Inequality.
- *Relaxed Problem*: Problem w. fewer restrictions. Cost of optimal solution to relaxed problem is admissible heuristic for original problem. (e.g. straight line distance for cities distances).
- *Dominance*: If $h_1(n) \geq h_2(n)$ for all n, h_1 dominates h_2. Dominant heuristics better for search if admissible. (e.g. straight line distance, vs trivial heuristic $h = 0$.)

Best-First Search

- **Frontier**: Priority queue ordered by evaluation function $f(n)$. Expands most promising node based on just $f(n)$ (no past cost).
- Not complete, not cost-optimal.

A* Search

- **Frontier**: Priority queue. (Best-First Search considering path cost).
- Use evaluation $f(n) = g(n) + h(n)$. $g(n)$ is cost to reach n from start. $h(n)$ is estimated cost to reach goal from n.
- **Complete**: if $h(n)$ is admissible, state space finite, step costs positive. **Optimal** if $h(n)$ is admissible.
- **Complexity**: Time: Exponential but improved by good heuristics. Space: Exponential. (Both $O(b^d)$ for poor heuristic.)

If $h(n)$ admissible, A* search (w/o visited memory) is optimal. If $h(n)$ consistent, A* search (w. visited memory) is optimal.

- A consistent heurisitic $\implies$ admissible (T2.QnC.1). Converse not true.
- (Trivial $h = 0$ consistent, admissible)
- Triangle inequality: Sum of lengths of any two sides $\geq$ length of 3^{rd} side.

3. Local Search

For (NP-hard) problems, traverse state space in systematic manner not option, is intractable (unsolvable in time).

Local Search: Explore state space considering only locally-reachable states ('search space'). Start random position, iterative move to neighboring states via perturbation or construction. Solution is final state, not path taken.
(E.g. N-Queens Problem, minimize function value)

- Typically incomplete and suboptimal.
- *Any-time property*: Longer runtime yields better solutions. Suitable for obtaining "good enough" solutions for difficult problems.
- **Types**: *Perturbative* Search (search space is complete solutions, modify components of current solution). *Constructive* Search (partial candidate solutions, extend by adding components).

Problem Formulation in Local Search:
States: Represent configurations/candidate sln within search space, but may not directly map to actual state. *Goal Test* (Optional) checks if current state is desired solution. *Successor Function*: Generates neighboring states by applying small modifications.
Evaluation/Objective Function: assess quality of state, max/minimize based on problem. (E.g. N-Queen Maximize "safe" queens, min. y value.)

Hill-Climbing Algorithm

Find local maxima by travelling to neighbouring states with highest value, terminate when no neighbour has higher value. One objective function is to negate heuristic function.

```
current_state = initial_state
while True:
    best_successor = highest-valued successor of curr_state
    if eval(best_successor) <= eval(current_state):
        return current_state
    current_state = best_successor
```

Greedy local search: gets stuck at local maxima, ridges, plateaus, shoulders (same values flat area). Some solutions include limited sideways move, stochastic (probability based on steepnes of move), first-choice, random-restart. Simulated Annealing - allow occasional 'bad moves' to escape local optima.

4. Adversarial Search and Games

'Classicial' Adversarial: Agents/Player compete against each other. Look at deterministic, two player, turn-taking, perfect information, zero-sum games. Perfect information: fully observable. Zero sum: no "win-win".

- *States*: Configurations of game, initial state, terminal state (win/lose/draw).
- *Actions*: Possible moves by a player.
- *Transition Model*: Defines how actions change the state.
- *Utility Function*: Assigns a value to terminal states from perspective of the agent. (E.g. Win: $+1$, Draw: 0, Loss: -1)

Minimax Algorithm

Determine optimal move for Player A (MAX) assuming Player B (MIN) plays optimally. Work out minimax value of each state in tree.

- **Complexity**: Time: Exponential w.r.t. max depth ($O(b^m)$). Space: Poly.
- **Complete**: If tree is finite. **Optimal** if against optimal opponent.

It is a recursive algo, backs up minimax values as recursion unwinds. Thus, similar to DFS, time c. $O(b^m)$, space c. $O(bm)$.
Note: minimax not 'optimal' against non-optimal MIN player, as non optimal player could choose higher value in unchosen path (which had lower min value). But guarantees it never lower utility than of optimal opponent.

Alpha-Beta Pruning (Minimax)

Reduces number of nodes (computational overhead) evaluated in game tree without affecting final decision. Minimax no need explore full tree. Good move ordering improves effectivness of pruning.

- α: MAX's highest guaranteed so far. MAX updates α.
- β: MIN's lowest guarantee so far. MIN updates β.
- At MAX node, prune (stop) if $\alpha \geq \beta$. At MIN node, prune (stop) if $\beta \leq \alpha$.
- Values of α, β passed down tree as explore nodes.
  When backtracking, values passed up to update α, β if applicable.

Good move ordering improves effectivness of pruning.

Evaluation Function to Cutoff (Minimax)

Fully exploring game tree with Minimax may be computationally infeasible.

- Apply **cutoff strategy** to halt the search at predefined maximum depth.
- Use **heuristic function** to estimate the utility of mid-game (non-terminal) states. Heuristic must be some value between win and loss, and computationally efficient. Admissibility, consistency properties do not apply. Can use domain-specific features or some labeled dataset to train heuristics.

5. Machine Learning

Computers with ability to learn without explicit programming, through algorithms that learn from, and make predictions based on data. Goal is for performance to improve over time by identifying patterns in the data.

Problem Types

- **Classification**: Output category value in finite set. (Predict discrete label)
- **Regression**: Output is number. (Predict continuous num. value)

Types of Machine Learning (Feedback difference)

1. **Supervised Learning**: Learns from labeled data (input-output pairs) to map inputs to outputs. (E.g. classify pic banana or apple)
2. **Unsupervised Learning**: Learns from unlabeled data to find patterns or structure. (E.g. clustering, group based on features).
3. **Semi-Supervised Learning**: Use labeled, unlabeled data for learning.
4. **Reinforcement Learning**: Learns to make decisions by interacting with an environment, (rewards and penalties).

Supervised Learning

- **Training Phase**: Learn to map inputs to outputs by minimizing difference btw. predictions and ground truth using a training set.
  Result of training called a *model / hypothesis*.
- **Testing/Evaluation Phase**: Measures the model's performance on unseen data (test set) to evaluate generalization.
- Supervised learning: dataset represented as set of pairs $(x^{(i)}, y^{(i)})$:
  - $x^{(i)} \in \mathbb{R}^d$: Input vector of i-th data point with d features.
  - $y^{(i)}$: Label (target) for the i-th data point. ($y^{(i)} \in \mathbb{R}$ or $\in \{1, \ldots, C\}$)
  - Dataset D with n examples: $D = \{(x^{(1)}, y^{(1)}) \ldots, (x^{(n)}, y^{(n)})\}$.

Assume unknown true relationship $y = f^*(x) + \epsilon$, ϵ noise error term. Find hypothesis $h(x)$ that approximates true function f^*.

- **Hypothesis class $\mathcal{H}$**: set of all possible learnable models or functions.
  Each $h \in \mathcal{H}$ is a hypothesis or model. (Classes - e.g decision tree, linear model, neural networks).
- **Learning Algorithms** A: takes training set D_{train}, find hypothesis $h \in \mathcal{H}$. (E.g. Decision tree learning, gradient descent algorithms)
- **Performance Measure**: Test hypothesis on new set (test set). Metrics:
  - (Regression) Mean Squared Error (MSE): $= \frac{1}{N}\sum_{i=1}^{N}(\hat{y}^{(i)} - y^{(i)})^2$.
  - (Regression) Mean Absolute Error (MAE): $= \frac{1}{N}\sum_{i=1}^{N}|\hat{y}^{(i)} - y^{(i)}|$.
  - (Classification) Accuracy: Accuracy $= \frac{1}{N}\sum_{i=1}^{N}\mathbf{1}(\hat{y}^{(i)} = y^{(i)})$.
  - (Classification) Confusion Matrix: e.g. Precision ($\frac{TP}{TP+FP}$ - maxm if FP costly), Recall ($\frac{TP}{TP+FN}$ - maxm if FN costly), F1 Score: $\frac{2}{\frac{1}{P}+\frac{1}{R}}$
- **Generalization**: How well model performs on unseen data. Affected by dataset quality/quantity (relevance, noise, balance), and Model Complexity. (size and expressiveness of hypothesis class - overfitting & underfitting). Simple model - high bias, low variance. Complex - low bias, high var.
- **Hyperparameter Tuning**: Adjust params (e.g. γ) to optimize performance. Set before training process begins (predefined, adj manually) - hyperparam search: grid (exhaustive), random, local, successive halving etc.

6. Decision Trees

Function that takes as input a vector of attribute values and returns a decision (single output value). Perform sequence of tests from root node to leaf.

- **Expressiveness**: Can express any (propositional) fn of input attributes.
- **Size of Hypothesis Class**: Distinct decision trees, n bool. attributes: 2^{2^n}
- **Decision Tree Learning**: Greedy, top-down, recursive algorithm. Use information gain, choose best attribute to divide training set. For each value of chosen attr. use DTL again on corresponding subset of examples.
- **Entropy**: $I(P(v_1), P(v_2), ..., P(v_k)) = -\sum_{i=1}^{k} P(v_i)\log_2 P(v_i)$
- **Information Gain**: Initial Entropy $-$ avg. entropy after divide 'remainder'. Entropy before dividing: $I(\frac{p}{p+n}, \frac{n}{p+n})$, $p+n$ examples in subset E_i.

Information Gain

Given *attribute* A with v distinct values, the training set is split into subsets $E_1, ..., E_v$, each with p_i positive and n_i negative examples.

- remainder$(A) = \sum_{i=1}^{v} \frac{p_i+n_i}{p+n} I\left(\frac{p_i}{p_i+n_i}, \frac{n_i}{p_i+n_i}\right)$
- Info Gain: $IG(A) = I(p_{\text{pos}}, p_{\text{neg}}) - \text{remainder}(A)$

Choose attribute with max information gain.
Generalisation/Overfitting: **Prune**, reduce size of decision tree.
• min-sample leaf (min threshold of sample count in leaf node)
• max depth (longest path from root to leaf) pruning.
Smaller DT may higher accuracy as sample due to noise ignored.
Data Preprocessing: Missing values (assign most common value, drop row/attribute etc.) Continuous values - partition to discrete set as attribute.

7. Linear Regression and Classification

Linear Model: Input vector x of dimension d, hypo class is set of linear fns:

$$h_w(x) = w_0x_0 + w_1x_1 + w_2x_2 + \cdots + w_dx_d = w^Tx.$$

where $w_0, w_1, \ldots, w_d$ are weights, $x_0 = 1$ is dummy variable (rmb to add)

$$\underset{\text{Design Matrix}}{X} = \begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & \ldots & x_d^{(1)} \\ 1 & x_1^{(2)} & x_2^{(2)} & \ldots & x_d^{(2)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & x_2^{(N)} & \ldots & x_d^{(N)} \end{bmatrix} \quad \underset{\text{Target Vector}}{Y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(N)} \end{bmatrix} \quad \underset{\text{Weight Vector}}{w} = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix} \in \mathbb{R}^{d+1}$$

- $X \in \mathbb{R}^{N\times(d+1)} | w \in \mathbb{R}^{d+1} | Y, \hat{Y} \in \mathbb{R}^N$ (target, predicted values).
- **Feature Transformations**: modify features for better modeling. Feature Engineering (e.g. poly $z = x^k$/ log $z = \log(x)$/ exp). Feature scaling (e.g. min-max scaling $z_i = \frac{x_i - \min(x_i)}{\max(x_i) - \min(x_i)}$, standardization $z_i = \frac{x_i - \mu_i}{\sigma_i}$).
- **Measure Fit**: Loss fn of weights w, find w to min. loss. (N : # train pts)
- **Loss Fn**: $J_{MSE}(w) = \frac{1}{2N}\sum_{i=1}^{N}(h_w(x^{(i)}) - y^{(i)})^2$.
- **Derivative**: $\frac{\partial J_{MSE}(w)}{\partial w_j} = \frac{1}{N}\sum_{i=1}^{N}\left(h_w(x^{(i)}) - y^{(i)}\right)x_j^{(i)}$

Learning via Normal Equation

- (MSE) Solve for w that minimizes J_{MSE}: $w = (X^TX)^{-1}X^TY$
- Problems: Computational cost: $O(d^3)$ for matrix inversion. Non-linear models cannot be solved using normal equations.

Learning via Gradient Descent

- **Algorithm**: Initialize w randomly. Update weights iteratively. Repeat until convergence: $w_j \leftarrow w_j - \gamma\frac{\partial J(w_0, w_1, \ldots)}{\partial w_j}$.
- **Learning rate** $\gamma > 0$: Controls step size. (Hyperparameter). can $\downarrow$ w time.
- **Variants**: Batch, use all data points. Mini-batch, use data subsets. Stochastic, uses one data point per iteration.
- **Convex**: MSE loss fn. convex, single global min, for linear/poly regression.

8. Logistic Regression and Classification

Logistic Regression Model: Hypo class: Set of "squashed" linear fns.

- **Sigmoid function**: $\sigma(x) = \frac{1}{1+e^{-x}}$
- Logistic model: $h_w(x) = \sigma(w_0x_0 + w_1x_1 + \cdots + w_dx_d) = \sigma(w^Tx)$ where $w_0, w_1, \ldots, w_d$ are weights, $x_0 = 1$ is a dummy variable.

Classification with Logistic Regression: Regression outputs value in $[0, 1]$, treat as probability of class 1. (E.g. $h_w(x) = 0.8$ predicts 80% chance of failure). Use **Decision threshold** (e.g. 0.5) for classification.
Desn. Boundary: surface (e.g. 2D line) separating classes in feature space.
Non-Linearly Separable Data: Non-linearly separable if single linear decision boundary cannot separate classes. Solutions: Use non-linear models or apply non-linear feature transformations.

Loss Function - Logistic Regression

MSE not ok for logistic due to non-convexity. Use **Binary Cross Entropy**.

- $BCE(y, \hat{y}) = -y\log(\hat{y}) - (1-y)\log(1-\hat{y})$, $|y \in \{0, 1\}$ and $\hat{y} \in [0, 1]$.
- Small loss if $y \approx \hat{y}$, large loss if y and $\hat{y}$ differ - good loss function.
- BCE loss is convex for logistic regression.
- Derivative: $\frac{\partial J_{BCE}(w)}{\partial w_j} = \frac{1}{N}\sum_{i=1}^{N}(h_w(x^{(i)}) - y^{(i)})x_j^{(i)}$. (same-linear)
- Weight update rule: $w_j \leftarrow w_j - \gamma\frac{\partial J_{BCE}(w)}{\partial w_j}$

Multi-Class Classification

Extends binary classification to multiple classes by breaking down into sets.
One-vs-one: Binary classifier for every pair of classes $total = \frac{C(C-1)}{2}$ classifiers. Class with most votes wins.
One-vs-rest: Binary classifier for each class, treat all other classes as combined class. C **classifiers.** Class with highest probability output wins.

9. Regularization, Kernels, Support Vector Machines

Regularization

Technique to ↓ overfitting, by **penalizing large weights in model**.

- Overfitting when model too complex, capturing training data noise over underlying pattern, leads to poor generalization on unseen data.
- Overfitting often associated w. models w. large weights. E.g.:
  - Complex polynomial model $h(x) = wx^3 - wx$ fits train data perfect, fail to generalize to new data. Large oscillations in model can occur as weights w increase, even while fitting the same set of points.

Regularization: **penalty term** to loss function, discourage large weights.

- Given a loss function $J(w)$ (e.g., MSE or BCE), add penalty function $P(w)$ scaled by a hyperparameter $\lambda \geq 0$, where **optimization goal** becomes:

$$J_{\text{reg}}(w) = J(w) + \lambda P(w), \text{ where we want: } \min_w J_{\text{reg}}(w). \text{(by varying } w)$$

- Perform feature scaling before regularization. (Same effect on all variables).

Common Penalty Functions

- **L2 penalty/regularizer (Ridge Regression)**: Penalizes square of weights, encourage smaller weights, smooths model.

$$P_{\text{square}}(w) = \frac{1}{2}\sum_{j=0}^{d} w_j^2, \quad P_{\text{abs}}(w) = \sum_{j=0}^{d} |w_j|, \quad P_{\max}(w) = \max(|w_0|...|w_d|)$$

- **L1 Regularizer (Lasso Regression)**: Penalizes absolute weight values, encourage sparsity, effectively feature selection by driving some weights to 0.
- **Max-Norm Penalty**: Constraint max weight, prevent extreme values.

Effect of Regularization on Learning Algorithms

- **Gradient Descent**: The gradient of the regularized loss function includes both the original loss gradient and the gradient of the penalty term:

$$\frac{\partial J_{\text{reg}}(w)}{\partial w_j} = \frac{\partial J(w)}{\partial w_j} + \lambda \frac{\partial P(w)}{\partial w_j}.$$

- **Normal Equation**: For linear regression with L2 regularization, the closed-form solution becomes:

$$w = (X^T X + \lambda I)^{-1} X^T Y,$$

where I is the identity matrix. This ensures invertibility of $X^T X + \lambda I$, even in cases of multicollinearity or insufficient data.

Trade-Off w. Regularization Strength λ: Hyperparam (penalty strength) λ controls trade-off between fitting training data and keeping weights small:

- Small λ: Emphasizes fitting the data, potentially leading to overfitting.
- Large λ: Emphasizes smaller weights, potentially leading to underfitting.

Kernel Method

Linear Combination of Vectors, Linear Model Weights & Training Data: A linear combination of vectors is sum of scalar multiples of those vectors. vector $\begin{bmatrix}2\\3\end{bmatrix} = 2\begin{bmatrix}1\\0\end{bmatrix} + 3\begin{bmatrix}0\\1\end{bmatrix}$. In linear regression, weight vector w can express as linear combo of training data points $x^{(j)}$, where coefficients α_j are real. Normal equation, $w = (X^T X)^{-1} X^T Y$ rewritten as $w = \sum_{j=1}^{N} \alpha_j x^{(j)}$. (The weights are derived from a weighted combination of training data.)

- **Dual Formulation of Linear Regression:** Hence, linear model $h_w(x) = w^T x$ can be rewritten in its dual form, where α_j are the new parameters (weights), and $x^{(j)}$ are training data points. This expresses predictions as weighted sum of dot products between training data $x^{(j)}$ and input x.

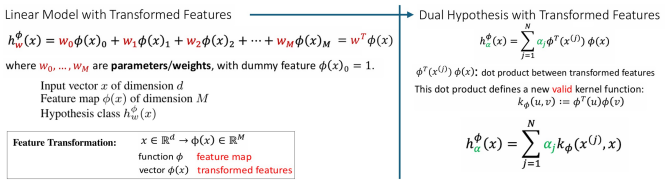
$$h_\alpha(x) = \sum_{j=1}^{N} \alpha_j (x^{(j)})^T x$$

- **Dot Product and Similarity**: Dot product $u \cdot v = u^T v$ measures similarity between 2 vectors. E.g. orthogonal vectors dot product of zero: $\begin{bmatrix}1\\1\end{bmatrix} \cdot \begin{bmatrix}1\\-1\end{bmatrix} = 0$

Dual Formulation with Kernel: Kernel Trick

Dual hypothesis dot product $(x^{(j)})^T x$ replace by kernel function $\mathbf{k}(\mathbf{x^{(j)}}, \mathbf{x})$:

$$h_\alpha(x) = \sum_{j=1}^{N} \alpha_j k(x^{(j)}, x).$$



Kernel function $k(u, v)$ generalizes dot product to measure similarity in potentially higher-dimensional space w/o explicit computing transformation.

- **Kernel Trick**: Replaces dot product with similarity function $k(u, v)$, enabling efficient computation of complex relationships in data. Valid kernel functions must satisfy conditions like symmetry and positive definiteness.
- A model that uses the kernel trick is called a **kernel machine**, which allows for implicit computation of high-dimensional feature transformations.
- **Examples of Kernels**:
  - **Polynomial Kernel**: $k(u, v) = (u^T v)^s$ where s is degree of polynomial.
  - **Gaussian (RBF) Kernel**: $k(u, v) = \exp\left(-\frac{\|u-v\|^2}{2\sigma^2}\right)$ where σ^2 controls width of Gaussian (↓ σ = narrower, ↑ = wider).
  - **String Kernel**: Measures similarity between strings.
  - **Chi-Squared Kernel**: Useful for histograms.
  - **Tanh Kernel**: Similar to neural network activation functions.
- **Transformed Features and Kernels**: Deep connection between kernels and feature transformations (Can be single variable, multi-variable transformations to create new features). Specifically:
  - Any valid kernel $k(u, v)$ corresponds to a feature map (transform) ϕ s.t.:

$$k(u, v) = \phi(u)^T \phi(v).$$

  - Conversely, any feature transformation ϕ induces a valid kernel.
- **Implicit, efficient** computation by Kernels of feature transformations, even when transformed feature space is infinite-dimensional. E.g.
  - The RBF kernel corresponds to an infinite-dimensional feature map.
  - Compute $\phi(x)$ explicitly maybe infeasible, evaluating $k(u, v)$ efficient.
- **E.g. Polynomial Feature Transform**: to all monomials of degree up to 2:

$$x = \begin{bmatrix}x_1\\x_2\end{bmatrix} \to \phi(x) = \begin{bmatrix}x_1\\x_2\\x_1^2\\x_2^2\\x_1x_2\end{bmatrix}, \text{corresponding kernel: } k(u, v) = (u^T v)^2.$$

Dual formulation of linear models expresses predictions as weighted sum of kernel evaluations. Kernels generalize dot product, enabling implicit computation of high/ infinite-dimensional feature transformations. The kernel trick is powerful for handling non-linear data efficiently.

Support Vector Machine (SVM)

SVM is a powerful classifier with several key aspects:

- **Optimal Decision Boundary**: SVM constructs an optimal separating decision boundary based on training data.
- **Fat Margin Classifier**: Maximizes the margin, which improves performance on unseen data and enhances robustness to noise.
- **Non-linear Decision Boundaries**: The kernel trick allows SVM to handle non-linear decision boundaries naturally.
- **Kernel Machine**: SVM is one of the most famous kernel machines.

Hyperplanes

The zero-offset hyperplane is defined by normal vector w and contains all vectors u s.t.: $w^T u = 0$.

- **Side**: Sign of $w^T u$ determines which side of hyperplane a point u lies:

$$\text{Sign}(w^T u) = \begin{cases} > 0 & \text{if } u \text{ is on one side,} \\ < 0 & \text{if } u \text{ is on the other side.} \end{cases}$$

- **Distance to Hyperplane:** Distance of a point u to hyperplane given by: Distance $= \frac{|w^T u|}{\|w\|}$ where $\|w\|$ is the length of the normal vector w.

SVM: Maximizing the Margin

- **Decision boundary** in SVM defined by hyperplane separating data points of different classes. The **margin** is distance between hyperplane and closest data points from each class. **Width of margin** proportional to 2×margin.
- **Optimization objective** is to maximize margin while ensuring all data points are correctly classified according to respective class.
- Data points that lie exactly on boundary of margin called **support vectors**. These points define position and orientation of hyperplane.

SVM: Data, Model, Objective, Sparsity

- Given N data points, described by features $x^{(i)}$, target $y^{(i)} \in \{-1, 1\}$.
- The SVM classifier in dual formulation depends on parameters α_i:

$$\text{SVM}_\alpha(x) = \text{sign}\left(\sum_{i=1}^{N} \alpha_i k(x^{(i)}, x)\right),$$

where $k(x^{(i)}, x)$ is the kernel function.

- The model is optimized by maximizing the margin subject to constraints that data points are on the correct side of the decision boundary.
- Learning algorithm: Quadratic programming.
- **Sparsity**: Many $\alpha_1, \alpha_2, \ldots, \alpha_N$ will be zero. Only support vectors contribute to classifier. Non-support vectors discarded without affecting model.

$$\text{SVM}_\alpha(x) = \text{sign}\left(\sum_{i \in \text{Support Vectors}} \alpha_i k(x^{(i)}, x)\right).$$

- **Implementation**: libraries e.g. `scikit-learn` to simplify implementation.

10. Unsupervised Learning, Cluster, Dimensionality Redn

Unsupervised Learning

- **Datasets w/o ground truth output**. (Unlike supervised - pairs of data point to target). Obtain ground truth expensive/ infeasible. E.g. images, videos w/o predefined categories of health, finance, preventative med w/o label.
- **Applications**: Clustering, dimensionality reduction, image segmentation, GPT (unsupervised learning component), anomaly detection etc.
- Given **set of** N **data points** $\{x_1, \ldots, x_N\}$, goal to learn patterns in data.

Clustering

Unsupervised machine learning technique to group similar data points into clusters/ groups. Goal to organize objects so those within same group more similar to each other than to in other groups.

- **Number of clusters** not defined by data.
- **Common applications:** Data segmentation, Anomaly detection
- **Approaches**: Partitioning-based clustering (K-means, K-Medoids (use actual data points - medoids - instead of centroids as cluster centers)), Hierarchical (agglomerative / divisive), Density-based Clustering (DBSCAN), Distribution-based, Spectral, Grid-based, Model based, Fuzzy etc.

K-Means Clustering

K-Means clustering - unsupervised learning algorithm to partition N data points into K distinct clusters. Each cluster represented by centroid, the mean of all points in the cluster.

K-Means Algorithm:

1. **Initialize**: Randomly initialize K centroids $\mu_1, \ldots, \mu_K$.

(a) **Assign each Point to Cluster**: $\forall x^{(i)}$, assign to nearest centroid: $c^{(i)} = \arg\min_k \|x^{(i)} - \mu_k\|^2$
($c^{(i)}$ is cluster assignment for $x^{(i)}$)

(b) **Update Centroids**: *Re*-compute centroids as mean of all points assigned to cluster: $\mu_k = \frac{1}{N_k} \sum_{i \in C_k} x^{(i)}$
(C_k is set of points in cluster k, N_k is number of points in C_k)

2. **Convergence**: The algorithm terminates when the assignments no longer change or the centroids stabilize.

- **Measuring the Quality of Clusters**:
The quality of the clusters can measure w. distortion function, computes average squared distance of each point to its assigned centroid:

$$J(c^{(1)}, c^{(2)}, \ldots, c^{(N)}, \mu_1, \mu_2, \ldots, \mu_K) = \frac{1}{N} \sum_{i=1}^{N} \|x^{(i)} - \mu_{c^{(i)}}\|^2.$$

- Each K-means step never increases distortion. (Convergence guaranteed).
- **Mean normalization, Feature Scaling** important to perform k-means clustering meaningfully (else cost function J dominated by certain feature.)
- **Local Optima**: K-Means prone to local opt. Use multiple random restarts.
- **Picking Clusters Count (K)**: K user-defined. Use **Elbow Method**, plot distortion J against K, "elbow" point where adding more clusters yields diminishing returns.
- **K-Means Variants**: *K-Medoids*: Actual data points as cluster reps (medoids). *Snap to Nearest Data Point*: Centroids "snapped" to nearest data points.
- **Distance-based Model**: Rely only on distance btwn points (vs. e.g. model).
- **Clustering vs. Classification**: Feedback (input/output): Unsupervised (clustering - distance based) vs. Supervised (classification - hyperplane based (e.g. SVM), probability based (logisitic reg)). Grouping/clustering for K-Means vs. a predictive model for classification. No .of classes defined by dataset (classification), no. of clusters defined by user (clustering).

Dimensionality Reduction

Technique to reduce number of features (variables or dimensions) in dataset while retaining as much relevant information as possible.

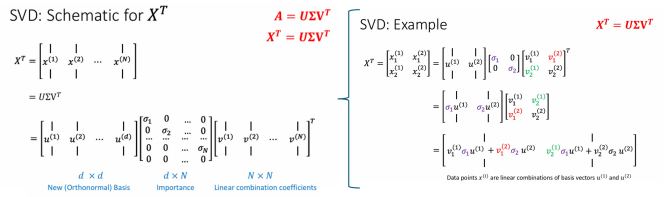
- **Curse of Dimensionality**: High-dimensional data (e.g. HD images - $1280 \times 720 = 921,600$ features) challenge in ML. Curse refers to exponential increase in sample complexity with the number of features: $N = O(2^d)$, d is no. of features, N is no. of samples required for effective learning.
- **Remove Redundant Features**: identified, removed by changing basis.

Singular Value Decomposition (SVD)

Matrix factorization technique / tool used in linear algebra, machine learning. Any real-valued matrix $A \in \mathbb{R}^{d \times N}$ decomposable into 3 components:

$$A = U \Sigma V^T,$$

- $U \in \mathbb{R}^{d \times d}$: Matrix of left singular vectors (new basis) w. orthonormal cols.
- $\Sigma \in \mathbb{R}^{d \times N}$: Diagonal matrix of singular values (importance of each basis vector). Singular values are ordered such that: $\sigma_1 \geq \ldots \geq \sigma_r \geq 0$.
- $V \in \mathbb{R}^{N \times N}$: Matrix of right singular vectors (coefficients for linear combinations) with orthonormal columns and rows.



- **Existence of SVD**: wlog, let $d > N$: for any $d \times N$ real-valued matrix A, there exists a factorization $A = U\Sigma V^T$.
- **Interpretation of SVD**: SVD provides new basis for data:
  - Columns of U form an orthonormal basis for the row space of A.
  - Columns of V form an orthonormal basis for the column space of A.
  - Singular values σ_j: importance of basis vector in capturing data variance.
- **Dimensionality Reduction via SVD**: As singular values σ_j indicate importance of corresponding basis vectors, to reduce dimensionality, retain only the top r singular values ($\sigma_1, \sigma_2, \ldots, \sigma_r$). Truncate Σ to $\tilde{\Sigma}$, keep only first r rows, columns. Use reduced basis matrix $\tilde{U}$ (first r columns of U) to project data into a lower-dimensional space.
- **Compression**: Original data matrix X^T approximated using truncated SVD:

$$\text{original: } X^T = U\Sigma V^T, \quad \text{so approx: } \tilde{X}^T = \tilde{U}\tilde{\Sigma}\tilde{V}^T$$

$$\tilde{X}^T = \tilde{U}Z = \tilde{U}\tilde{U}^T X^T \approx X^T. \quad (\tilde{U}\tilde{U}^T \approx I)$$

where $\tilde{U} \in \mathbb{R}^{d \times r}$, $\tilde{\Sigma} \in \mathbb{R}^{r \times r}$, and $\tilde{V} \in \mathbb{R}^{N \times r}$, $(X^T / \tilde{X}^T \in \mathbb{R}^{d \times N})$.
where $\tilde{U}$: reduced left singular vectors, Z: compressed rep of data.

- **Example: Data Compression**: Hence, given data matrix X^T, we can compress data by projecting it onto new reduced (lower-dimensional) basis $\tilde{U}$:

$$Z = \tilde{U}^T X^T, \quad Z \in \mathbb{R}^{r \times N}$$

Intuition: Project data points $x^{(i)}$. Instead of explicitly working w. $\tilde{\Sigma}$, $\tilde{V}^T$, project data onto reduced basis $\tilde{U}$. Z contains all projection coefficients.

- **Assumption of Mean-Centered Data**: Data matrix X^T assumed to be mean-centered (each feature has zero mean). Ensure singular values σ_j^2 directly represent variances along principal directions (axes) defined by singular vectors $u^{(j)}$, simplify interpretation of SVD results.
- **Reconstruction Error**: Error depends on retained singular values. To retain at least 99% of variance, choose smallest r such that:

$$\text{Retained Variance} = \frac{\sum_{i=1}^{r} \sigma_i^2}{\sum_{i=1}^{N} \sigma_i^2} \geq 0.99.$$

- **Applications of SVD**: Dimensionality reduction while retain variance. Noise reduction, remove less impt singular values, filter out noise. Data visualization: Projecting data into 2D or 3D for visualization. Recommendation systems: use SVD to uncover latent factors in user-item interaction matrices.
- **Practical implementation**: SVD available using popular numerical computing libraries (*Numpy*: `numpy.linalg.svd`, *Matlab*: `svd`)

Principal Component Analysis (PCA)

PCA is a statistical application of SVD. PCA identifies directions (principal components) that capture maximum variance in data. Principal components correspond to largest singular values in SVD. By projecting data onto top r principal components, PCA achieves dimensionality reduction while retaining most important information.